

```

import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertFalse;
import static org.junit.Assert.assertTrue;

import org.junit.Test;

import components.set.Set;

/**
 * JUnit test fixture for {@code Set<String>}s constructor and kernel methods.
 *
 * @author cl_c2231am03 (Tyler zhuangzhuang He & Meng Zhang)
 */
public abstract class SetTest {

    /**
     * Invokes the appropriate {@code Set} constructor for the implementation
     * under test and returns the result.
     *
     * @return the new set
     * @ensures constructorTest = {}
     */
    protected abstract Set<String> constructorTest();

    /**
     * Invokes the appropriate {@code Set} constructor for the reference
     * implementation and returns the result.
     *
     * @return the new set
     * @ensures constructorRef = {}
     */
    protected abstract Set<String> constructorRef();

    /**
     * Creates and returns a {@code Set<String>} of the implementation under
     * test type with the given entries.
     *
     * @param args
     *         the entries for the set
     * @return the constructed set
     * @requires [every entry in args is unique]
     * @ensures createFromArgsTest = [entries in args]
     */
    private Set<String> createFromArgsTest(String... args) {
        Set<String> set = this.constructorTest();
        for (String s : args) {
            assert !set.contains(s) : ""
                + "Violation of: every entry in args is unique";
            set.add(s);
        }
        return set;
    }

    /**
     * Creates and returns a {@code Set<String>} of the reference implementation
     * type with the given entries.
     *
     * @param args
     *         the entries for the set
     * @return the constructed set
     * @requires [every entry in args is unique]
     * @ensures createFromArgsRef = [entries in args]
     */
    private Set<String> createFromArgsRef(String... args) {
        Set<String> set = this.constructorRef();
        for (String s : args) {
            assert !set.contains(s) : ""
                + "Violation of: every entry in args is unique";

```

```
        set.add(s);
    }
    return set;
}

// test cases for constructor
@Test
public final void testConstructor() {
    Set<String> set = this.constructorTest();
    Set<String> expected = this.constructorRef();

    assertEquals(expected, set);
}

@Test
public final void testConstructorSize() {
    Set<String> set = this.constructorTest();

    assertEquals(0, set.size());
}

// test cases for add
@Test
public final void testAddToEmpty() {
    Set<String> set = this.constructorTest();
    Set<String> expected = this.createFromArgsRef("B");

    set.add("B");

    assertEquals(expected, set);
    assertEquals(1, set.size());
}

@Test
public final void testAddToNonEmpty() {
    Set<String> set = this.createFromArgsTest("A", "B");
    Set<String> expected = this.createFromArgsRef("A", "B", "C");

    set.add("C");

    assertEquals(expected, set);
}

@Test
public final void testAddToLargerSet() {
    Set<String> set = this.createFromArgsTest("A", "B", "C", "D", "E", "F");
    Set<String> expected = this.createFromArgsRef("A", "B", "C", "D",
        "E", "F", "G");

    set.add("G");

    assertEquals(expected, set);
}

// test cases for remove
@Test
public final void testRemoveFromOnlyOneElement() {
    Set<String> set = this.createFromArgsTest("A");
    Set<String> expected = this.constructorRef();

    String removed = set.remove("A");

    assertEquals("A", removed);
    assertEquals(expected, set);
}

@Test
public final void testRemoveFromMultiElement() {
    Set<String> set = this.createFromArgsTest("A", "B", "C");
```

```
        Set<String> expected = this.createFromArgsRef("A", "C");

        String removed = set.remove("B");

        assertEquals("B", removed);
        assertEquals(expected, set);
    }

    @Test
    public final void testRemoveRootWithTwoChildren() {
        Set<String> set = this.createFromArgsTest("D", "B", "F", "A", "C",
            "E", "G");
        Set<String> expected = this.createFromArgsRef("B", "F", "A", "C",
            "E", "G");

        String removed = set.remove("D");

        assertEquals("D", removed);
        assertEquals(expected, set);
    }

    @Test
    public final void testRemoveLeaf() {
        Set<String> set = this.createFromArgsTest("D", "B", "F", "A", "C",
            "E", "G");
        Set<String> expected = this.createFromArgsRef("D", "B", "F", "C",
            "E", "G");

        String removed = set.remove("A");

        assertEquals("A", removed);
        assertEquals(expected, set);
    }

    @Test
    public final void testRemoveNodeWithOnlyLeftChild() {
        Set<String> set = this.createFromArgsTest("D", "B", "F", "A");
        Set<String> expected = this.createFromArgsRef("D", "A", "F");

        String removed = set.remove("B");

        assertEquals("B", removed);
        assertEquals(expected, set);
    }

    @Test
    public final void testRemoveNodeWithOnlyRightChild() {
        Set<String> set = this.createFromArgsTest("D", "B", "F", "C");
        Set<String> expected = this.createFromArgsRef("D", "C", "F");

        String removed = set.remove("B");

        assertEquals("B", removed);
        assertEquals(expected, set);
    }

    // test cases for removeAny
    @Test
    public final void testRemoveAnyFromOnlyOneElement() {
        Set<String> set = this.createFromArgsTest("A");
        Set<String> expected = this.createFromArgsRef("A");

        String removed = set.removeAny();

        assertEquals("A", removed);
        assertEquals(0, set.size());

        set.add("A");
        assertEquals(expected, set);
    }
}
```

```
        assertEquals(expected.size(), set.size());
    }

    @Test
    public final void testRemoveAnyFromMultiElement() {
        Set<String> set = this.createFromArgsTest("A", "B", "C", "D");
        Set<String> expected = this.createFromArgsRef("A", "B", "C", "D");

        String removed = set.removeAny();
        assertEquals(3, set.size());
        assertFalse(set.contains(removed));

        set.add(removed);
        assertEquals(expected, set);
        assertEquals(expected.size(), set.size());
    }

    // test cases for contains
    @Test
    public final void testContainsLeftTrue() {
        Set<String> set = this.createFromArgsTest("A", "B", "C", "D");
        Set<String> expected = this.createFromArgsRef("A", "B", "C", "D");

        boolean contain = set.contains("A");

        assertTrue(contain);
        assertEquals(expected, set);
    }

    @Test
    public final void testContainsRightTrue() {
        Set<String> set = this.createFromArgsTest("A", "B", "C", "D");
        Set<String> expected = this.createFromArgsRef("A", "B", "C", "D");

        boolean contain = set.contains("D");

        assertTrue(contain);
        assertEquals(expected, set);
    }

    @Test
    public final void testContainsMiddleTrue() {
        Set<String> set = this.createFromArgsTest("A", "B", "C", "D");
        Set<String> expected = this.createFromArgsRef("A", "B", "C", "D");

        boolean contain = set.contains("C");

        assertTrue(contain);
        assertEquals(expected, set);
    }

    @Test
    public final void testContainsFalse() {
        Set<String> set = this.createFromArgsTest("A", "B", "C", "D");
        Set<String> expected = this.createFromArgsRef("A", "B", "C", "D");

        boolean contain = set.contains("Z");

        assertFalse(contain);
        assertEquals(expected, set);
    }

    @Test
    public final void testContainsEmptyFalse() {
        Set<String> set = this.createFromArgsTest();
        Set<String> expected = this.constructorRef();

        boolean contain = set.contains("A");
```

```
        assertFalse(contain);
        assertEquals(expected, set);
    }

    // test cases for size
    @Test
    public final void testSizeEmpty() {
        Set<String> set = this.createFromArgsTest();

        assertEquals(0, set.size());
    }

    @Test
    public final void testSizeOne() {
        Set<String> set = this.createFromArgsTest("A");

        assertEquals(1, set.size());
    }

    @Test
    public final void testSizeMany() {
        Set<String> set = this.createFromArgsTest("A", "B", "C", "D");

        assertEquals(4, set.size());
    }
}
```